AI를 활용한 상권추천 서비스

~~Code~~lab 아카데미

GitHub 프론트엔드(클릭)
백엔드(클릭)

시연 영상(클릭)

TEAM 2 : 차자조
AI
이현무, 이주봉(팀장), 배태민, 황대윤, 김호현
[멘토] 임정원

목차

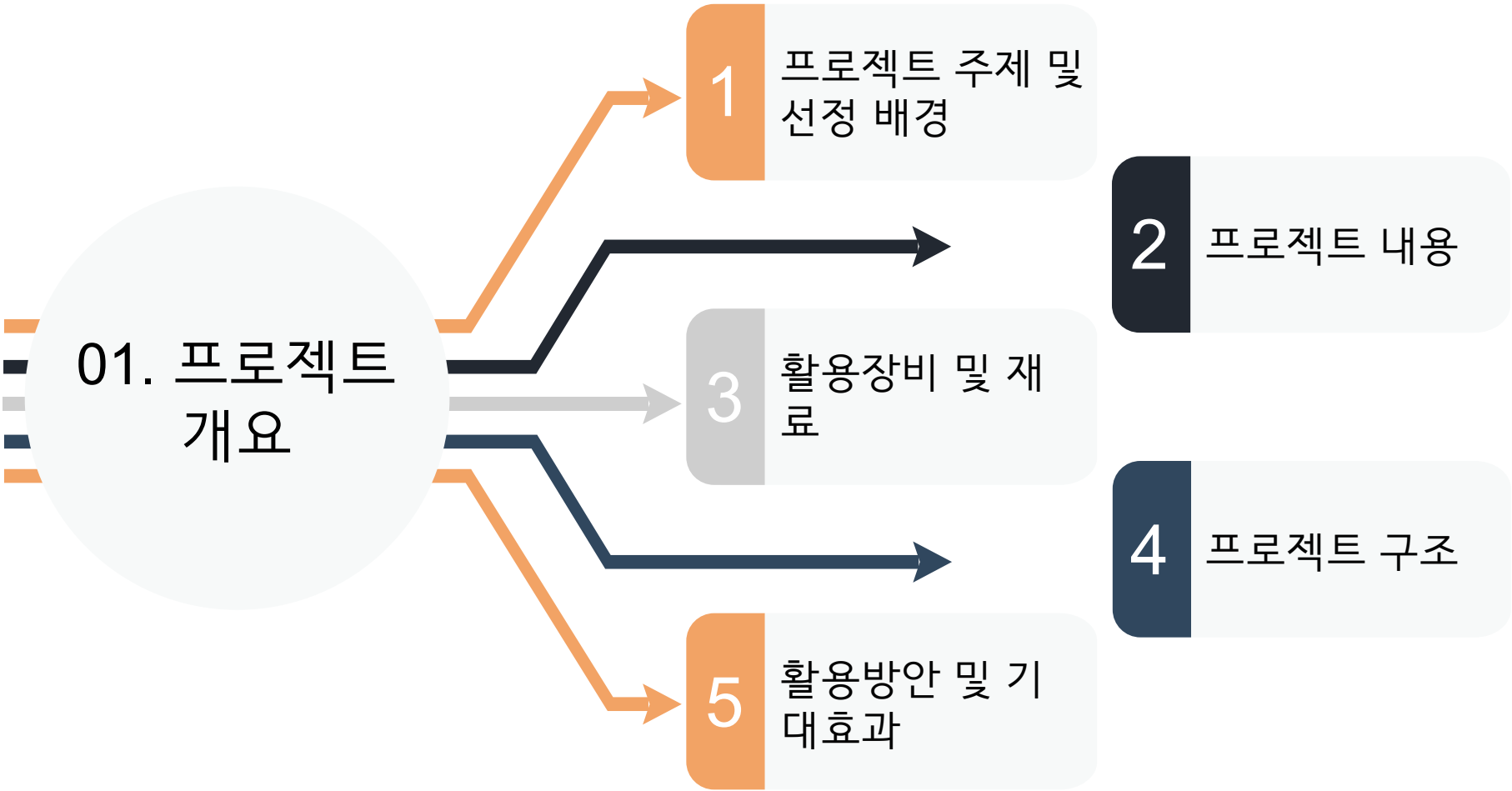
01
프로젝트 개요

02
프로젝트 팀 구성 및 역할

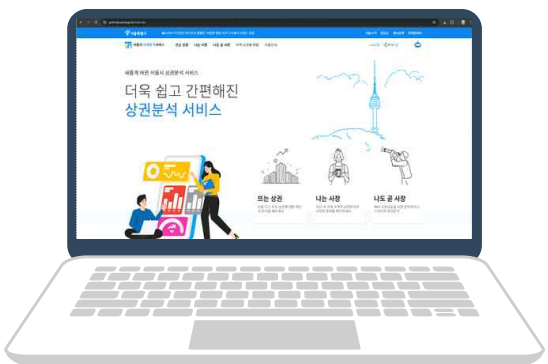
03
프로젝트 수행 절차 및 방법

04
프로젝트 수행 결과

05
자체 평가 의견


01. 프로젝트 개요
1
프로젝트 주제 및 선정 배경
2
프로젝트 내용
3
활용장비 및 재료
4
프로젝트 구조
5
활용방안 및 기대효과

1 프로젝트 주제 및 선정 배경



프로젝트 주제

서울시 행정구, 동 단위 업종별 창업 추천 서비스

선정 배경

정보의 홍수 속에서도, 정보의 불균형으로 인한 차이가 발생하고 있는 상황에서, 창업을 준비하는 예비 사장님들의 의사결정에 도움을 주기 위하여 기획

기획 의도

서울시 공공 데이터 등의 과거 데이터를 활용하여 창업 추천 서비스를 제공함으로써 창업 의사 결정을 지원

<참고사이트>

https://golmok.seoul.go.kr/main.do

https://bigdata.sbiz.or.kr/#/

2 프로젝트 내용

AI 모델

서울시 공공데이터를 기반으로, 행정구/동 단위의 업종별 창업추천 AI모델을 구현

프론트엔드

React + VITE 를 활용한 프론트엔드 구현

백엔드

파이썬 언어를 활용한 Data EDA, ML 모델링 및 FastAPI 구현

3 활용장비 및 재료

Client

HTML 5, JS 5, CSS 3

Vite + React

Server

FastAPI, PyMySQL, AXIOS

Data

MySQL, MySQL Workbench, ERD, QGIS

API

kakaomap, 서울 열린데이터 광장, DATA.GO.KR 공공데이터포털, nivo

Deployment

docker, docker Compose

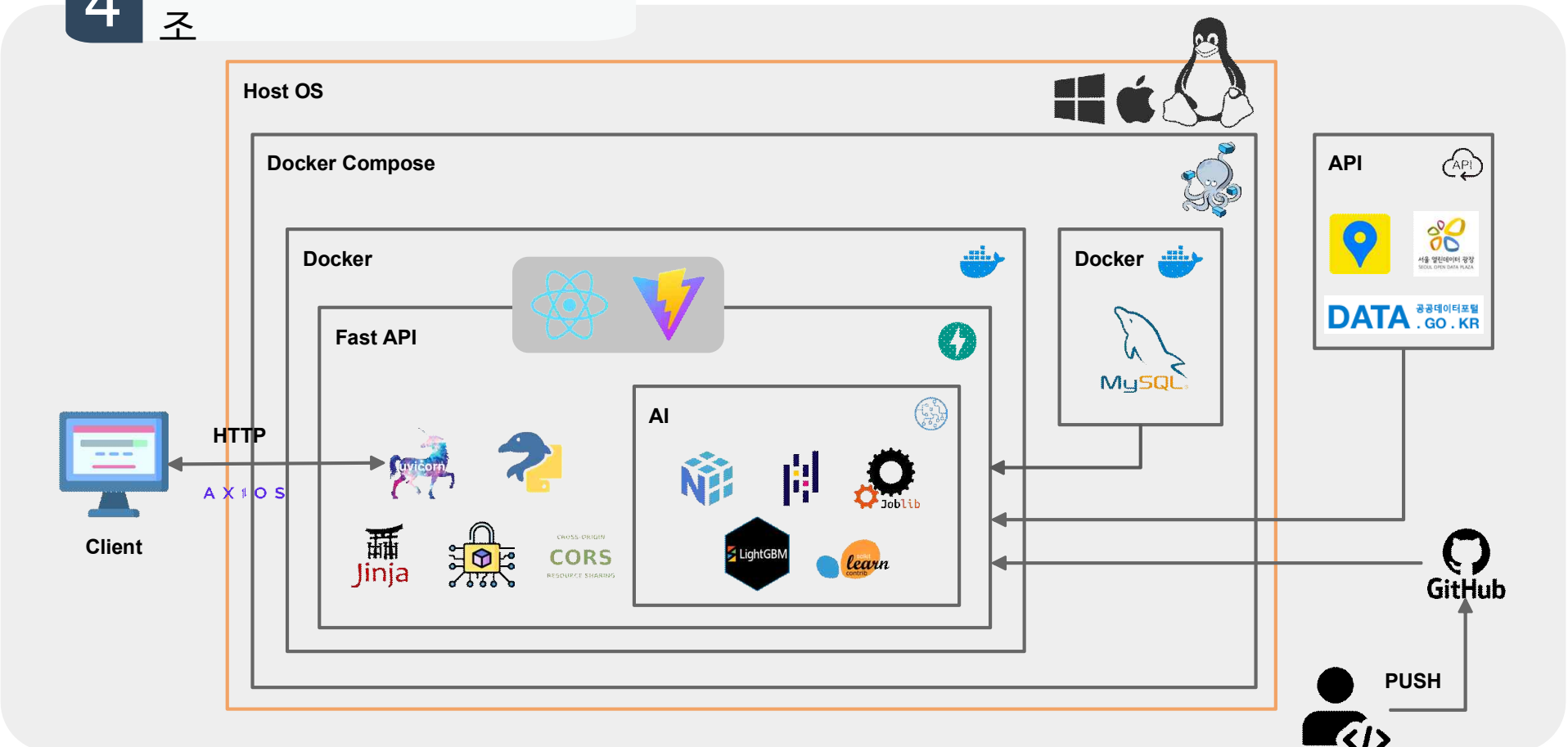
ML

jupyter, Jupyter, Google colab, scikit learn, LightGBM, ANACONDA

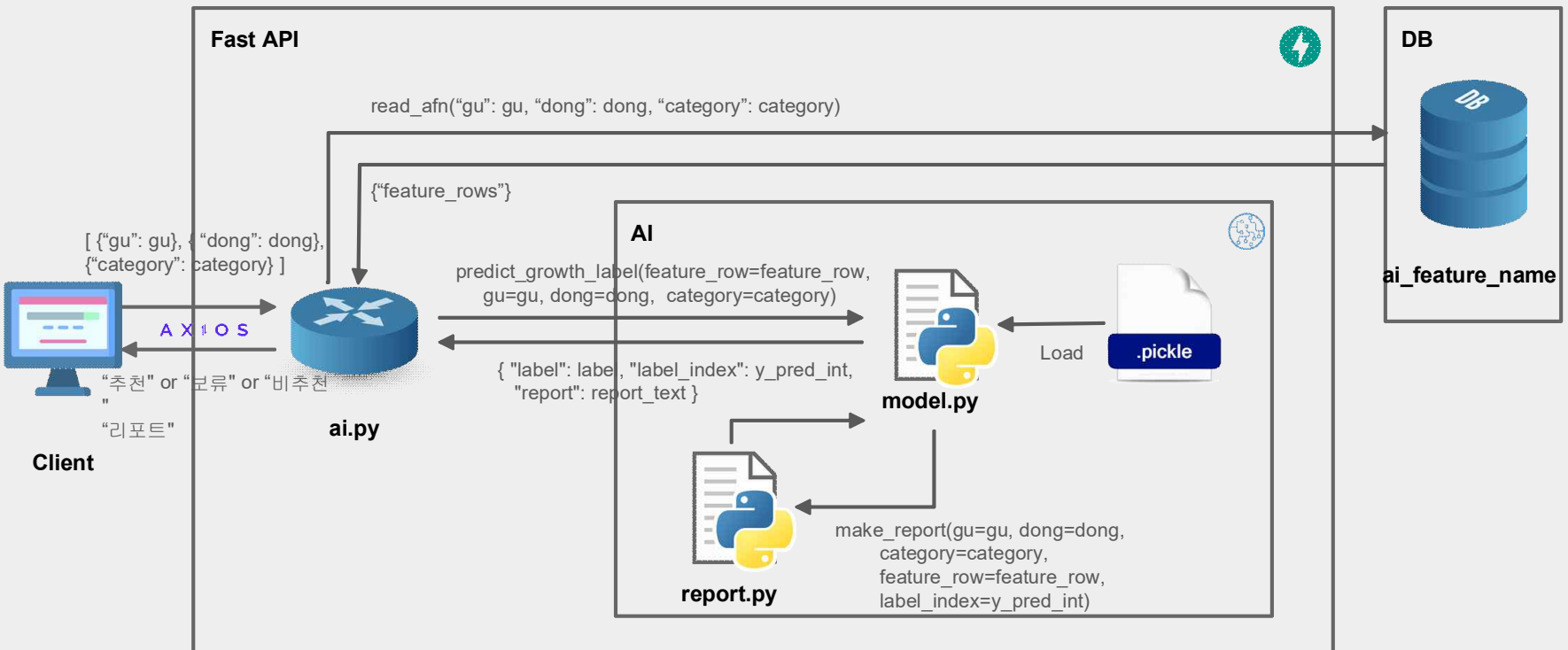
Co-work

GitHub, slack, Notion, Swagger

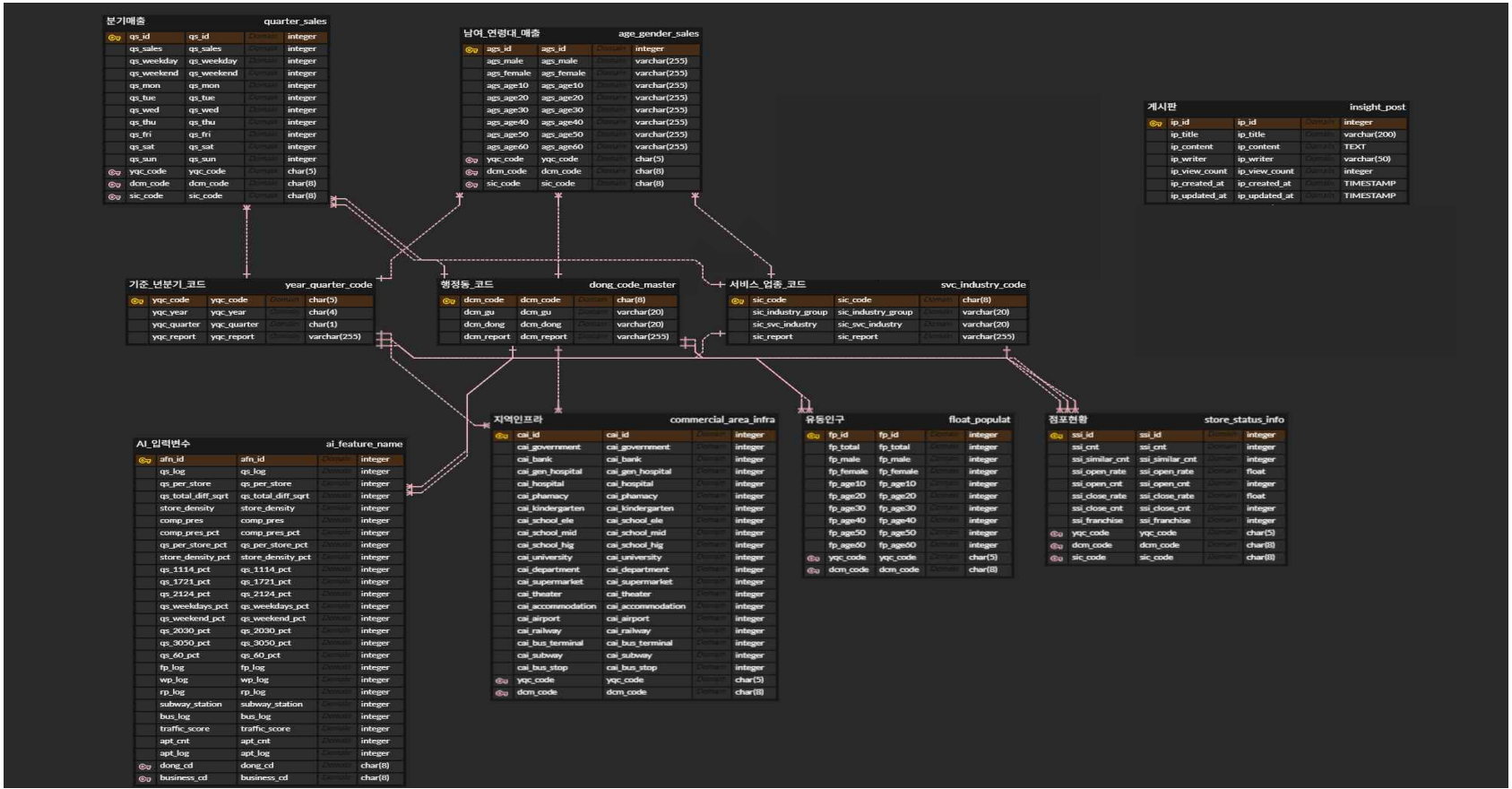
4 프로젝트 구조: 시스템 구조



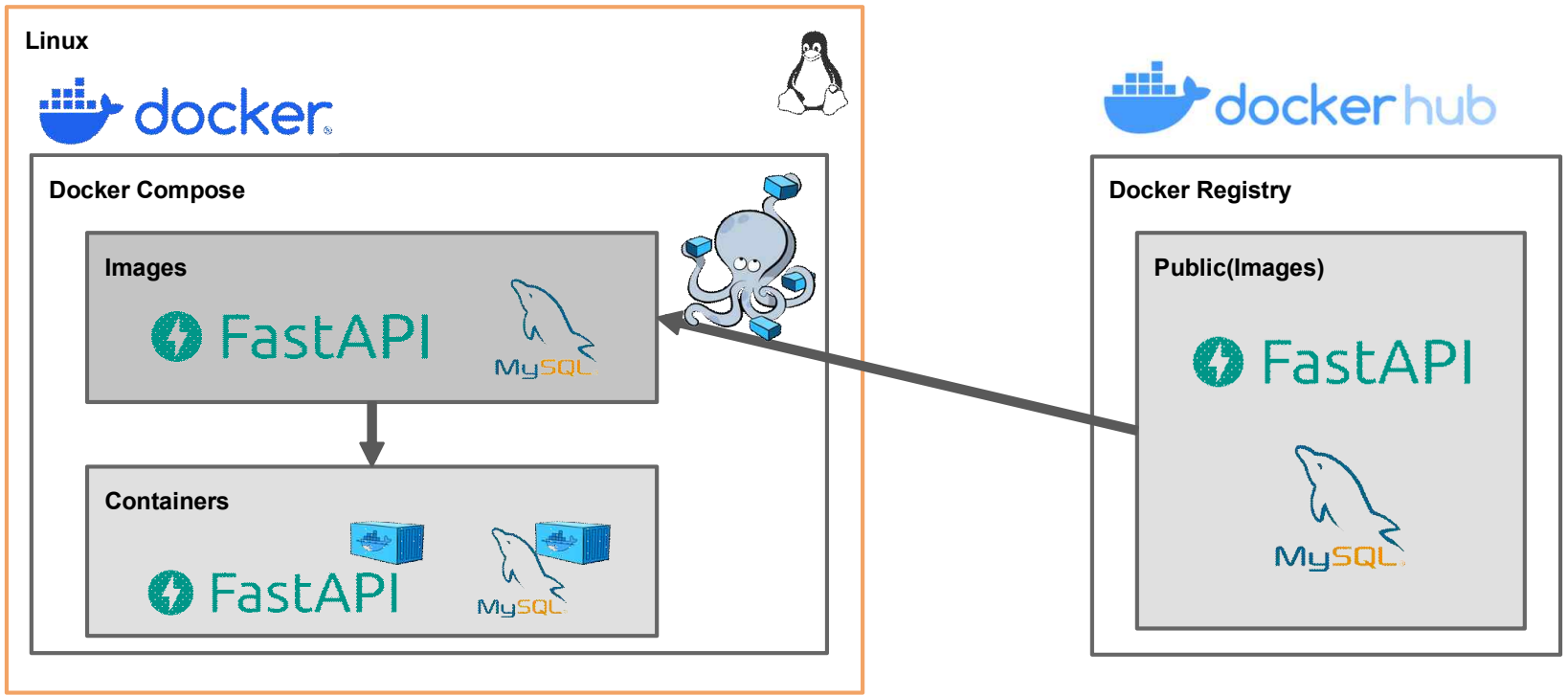
4 프로젝트 구조: AI 구조



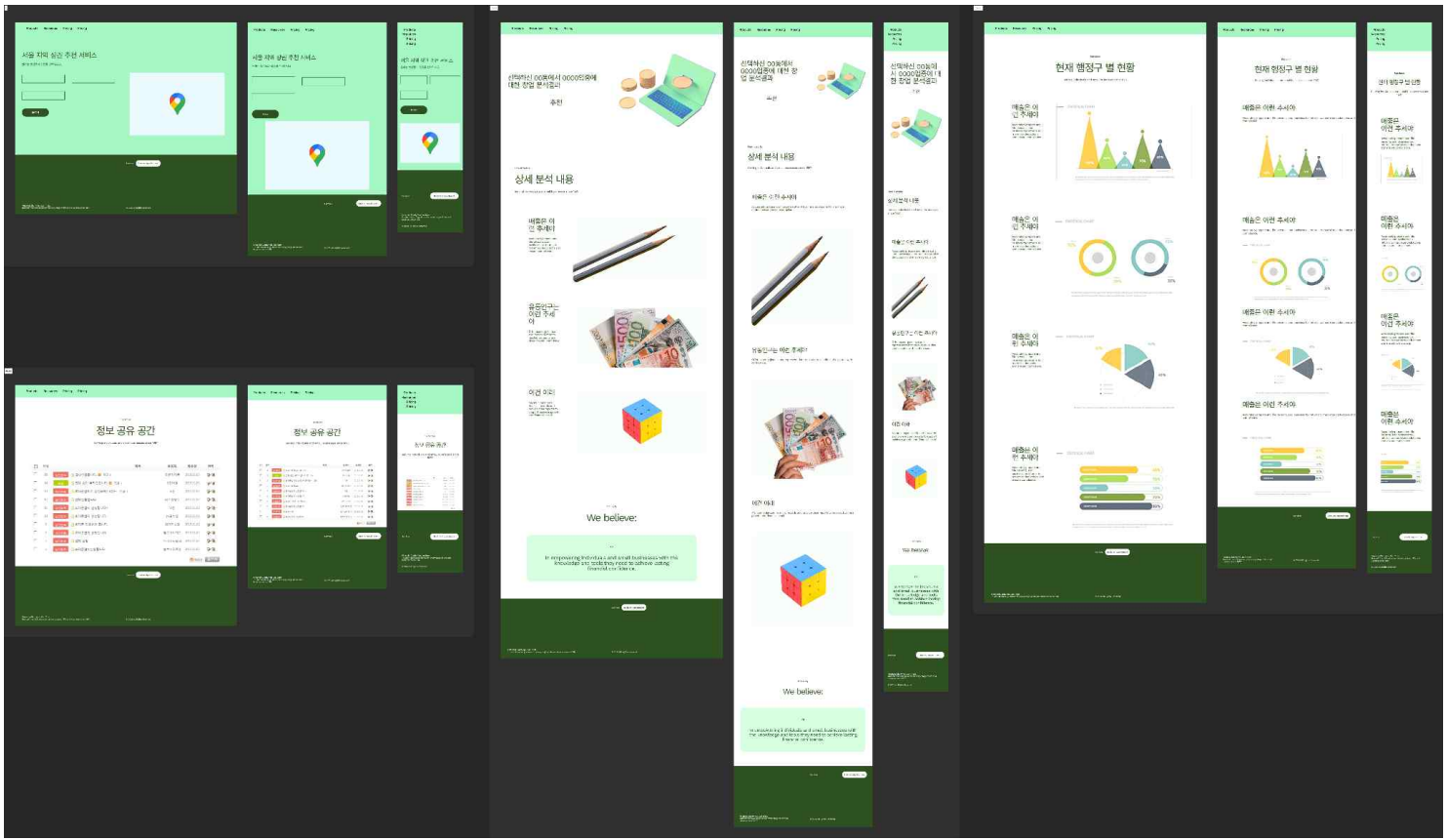
4 프로젝트 구조: ERD


분기매출 quarter_sales
qs_id qs_id integer
qs_sales qs_sales integer
qs_weekday qs_weekday integer
qs_weekend qs_weekend integer
qs_mon qs_mon integer
qs_tue qs_tue integer
qs_wed qs_wed integer
qs_thu qs_thu integer
qs_fri qs_fri integer
qs_sat qs_sat integer
qs_sun qs_sun integer
yqc_code yqc_code char(5)
dcm_code dcm_code char(8)
sic_code sic_code char(8)
남여_연령대_매출 age_gender_sales
ags_id ags_id integer
ags_male ags_male varchar(255)
ags_female ags_female varchar(255)
ags_age10 ags_age10 varchar(255)
ags_age20 ags_age20 varchar(255)
ags_age30 ags_age30 varchar(255)
ags_age40 ags_age40 varchar(255)
ags_age50 ags_age50 varchar(255)
ags_age60 ags_age60 varchar(255)
yqc_code yqc_code char(5)
dcm_code dcm_code char(8)
sic_code sic_code char(8)
게시판 insight_post
ip_id ip_id integer
ip_title ip_title varchar(200)
ip_content ip_content TEXT
ip_writer ip_writer varchar(50)
ip_view_count ip_view_count integer
ip_created_at ip_created_at TIMESTAMP
ip_updated_at ip_updated_at TIMESTAMP
기준_년분기_코드 year_quarter_code
yqc_code yqc_code char(5)
yqc_year yqc_year char(4)
yqc_quarter yqc_quarter char(1)
yqc_report yqc_report varchar(255)
행정동_코드 dong_code_master
dcm_code dcm_code char(8)
dcm_gu dcm_gu varchar(20)
dcm_dong dcm_dong varchar(20)
dcm_report dcm_report varchar(255)
서비스_업종_코드 svc_industry_code
sic_code sic_code char(8)
sic_industry_group sic_industry_group varchar(20)
sic_svc_industry sic_svc_industry varchar(20)
sic_report sic_report varchar(255)
AI_입력변수 ai_feature_name
afn_id afn_id integer
qs_log qs_log integer
qs_per_store qs_per_store integer
qs_total_diff_sqrt qs_total_diff_sqrt integer
store_density store_density integer
comp_pres comp_pres integer
comp_pres_pct comp_pres_pct integer
qs_per_store_pct qs_per_store_pct integer
store_density_pct store_density_pct integer
qs_1114_pct qs_1114_pct integer
qs_1721_pct qs_1721_pct integer
qs_2124_pct qs_2124_pct integer
qs_weekdays_pct qs_weekdays_pct integer
qs_weekend_pct qs_weekend_pct integer
qs_2030_pct qs_2030_pct integer
qs_3050_pct qs_3050_pct integer
qs_60_pct qs_60_pct integer
fp_log fp_log integer
wp_log wp_log integer
rp_log rp_log integer
subway_station subway_station integer
bus_log bus_log integer
traffic_score traffic_score integer
apt_cnt apt_cnt integer
apt_log apt_log integer
dong_cd dong_cd char(8)
business_cd business_cd char(8)
지역인프라 commercial_area_infra
cai_id cai_id integer
cai_government cai_government integer
cai_bank cai_bank integer
cai_gen_hospital cai_gen_hospital integer
cai_hospital cai_hospital integer
cai_phamacy cai_phamacy integer
cai_kindergarten cai_kindergarten integer
cai_school_ele cai_school_ele integer
cai_school_mid cai_school_mid integer
cai_school_hig cai_school_hig integer
cai_university cai_university integer
cai_department cai_department integer
cai_supermarket cai_supermarket integer
cai_theater cai_theater integer
cai_accommodation cai_accommodation integer
cai_airport cai_airport integer
cai_railway cai_railway integer
cai_bus_terminal cai_bus_terminal integer
cai_subway cai_subway integer
cai_bus_stop cai_bus_stop integer
yqc_code yqc_code char(5)
dcm_code dcm_code char(8)
유동인구 float_populat
fp_id fp_id integer
fp_total fp_total integer
fp_male fp_male integer
fp_female fp_female integer
fp_age10 fp_age10 integer
fp_age20 fp_age20 integer
fp_age30 fp_age30 integer
fp_age40 fp_age40 integer
fp_age50 fp_age50 integer
fp_age60 fp_age60 integer
yqc_code yqc_code char(5)
dcm_code dcm_code char(8)
점포현황 store_status_info
ssi_id ssi_id integer
ssi_cnt ssi_cnt integer
ssi_similar_cnt ssi_similar_cnt integer
ssi_open_rate ssi_open_rate float
ssi_open_cnt ssi_open_cnt integer
ssi_close_rate ssi_close_rate float
ssi_close_cnt ssi_close_cnt integer
ssi_franchise ssi_franchise integer
yqc_code yqc_code char(5)
dcm_code dcm_code char(8)
sic_code sic_code char(8)

4 프로젝트 구조: 배포


Linux
docker
Docker Compose
Images
FastAPI
MySQL
Containers
FastAPI
MySQL
dockerhub
Docker Registry
Public(Images)
FastAPI
MySQL

4 프로젝트 구조: UI



4 프로젝트 구조: API 명세서

1	요구사항 정의서								
2	db명 :								
3	Base URL : http://localhost:8000								
4			요청			라우터(각 구현하는 팀원별 라우터 파일 별도로 생성)		db (각 구현하는 팀원별 db처리 파일 별도로 생성)	db-table
5	기능(=use case)	담당자	요청방식	요청전달데이터	URI	응답내용	라우터 파일이름		
6	AI	황대윤, 김호현							
7	AI 결과조회		GET	{"gu":" ","dong":" ","category":" "}	/ai	{"request":{"gu":" ","dong":" ","category":" "},"prediction":{"label":" ","label_index":" ","report":" "}}	ai.py	dbt_afn.py	dong_code_master/ sic_svc_industry / ai_feature_name
8	시각화1(결과)	이주봉							
9	구, 동 검색		GET	gu	/gu	['구1', '구2',...,'구3'] ['동1', '동2',...,'동3']	gu.py	dbt_dcm.py	dong_code_master
10	업종검색		GET	category	/category	['카1', '카2',...,'카3']	category.py	dbt_sic.py	svc_industry_code
11	상세조회		GET	dcm_code, category	/result	[{ }, { }, { }, ..., { }]	result.py	dbt_result.py	quarter_sales / age_gender_sales / commercial_area_infra / float_populat / store_status_info
12	시각화2(현황)	황대윤							
13	서울 자치구 현황		GET		/status	{{ }, { }, { }, { }}	status.py	status_db.py	dong_code_master / year_quarter_code / quarter_sales / float_populat / store_status_info
14	자치구 현황 검색		GET		/status_map	{{ }, { }, { }, { }}	status_map.py	status_map_db.py	dong_code_master / year_quarter_code / svc_industry_code / quarter_sales / float_populat / store_status_info / age_gender_sales
15	게시판	김호현							
16	게시판 목록		GET	query(page:int=1, size:int=10, keyword:str?, writer:str?)	/board	{"items":[{"ip_id":int,"ip_title":str,"ip_writer":str,"ip_view_count":int,"ip_created_at":str}], "page":int, "size":int, "total":int}	board.py	dbt_ip.py	insight_post
17	게시판 작성		POST	body({"ip_title":str,"ip_content":str,"ip_writer":str})		{"message":"created","data":{"ip_id":int,"ip_title":str,"ip_content":str,"ip_writer":str,"ip_view_count":0,"ip_created_at":str}}			
18	글보기		GET	path(ip_id:int)		{"ip_id":int,"ip_title":str,"ip_content":str,"ip_writer":str,"ip_view_count":int,"ip_created_at":str}			
19	게시판 수정		PUT	path(ip_id:int) + body({"ip_title":str?,"ip_content":str?})	/board{ip_id}	{"message":"updated","data":{"ip_id":int,"ip_title":str,"ip_content":str,"ip_writer":str,"ip_view_count":int,"ip_created_at":str}}			
20	게시판 삭제		DELETE	path(ip_id:int)		{"message":"deleted","ip_id":int}			

4 프로젝트 구조: Swagger

FastAPI 0.1.0 OAS 3.1

/openapi.json

AI

- GET /ai/ AI Predict

GU

- GET /gu/ Select Gu
- GET /gu/dong Select Dong

Category

- GET /category/ Select Category

Result

- GET /result/ Result

Status

- GET /status/ Status

StatusMap

- GET /status_map/ 서울 25개 구 GeoJSON
- GET /status_map/gu_info/{gu_code} Gu Info
- GET /status_map/gu_state Gu State

Board

- GET /board/ Select Bd
- GET /board/{in_id} Select Bd One
- DELETE /board/{in_id} Delete Post
- PUT /board/{in_id} Update Post
- POST /board/write Write Bd One

default

- GET / Root
- GET /.well-known/appspecific/com.chrome.devtools.json Chrome Devtools Config

5 활용방안 및 기대효과

창업 계획 시, 원하는 지역에
가 하고자 하는 업종이 적합한지 인
트를 얻을 수 있음

데이터 접근성이 다소 떨어지는
계층에게 손쉽게 다양한 정보를
제공

창업 지원 및 프랜차이즈 입지 선정
등에 활용 가능

02. 프로젝트 팀 구성 및 역할

팀 구성 및 역할

이현무

- 가상화 백엔드 구성
- DB구조 설계 및 관리
- DB 정규화
- Github 운영 및 관리
- 데이터 학습 모델 검증

이주봉

- 프로젝트 관리
- 원시데이터 취합 및가공
- 프론트엔드 구조
- AI 분류 결과 검색 시각화 프론트/백엔드 구현
- 데이터 학습 모델 검증

배태민

- 일정관리
- 분류 데이터 전처리 및 EDA
- 게시판 프론트/백엔드 구현 지원
- 데이터 학습 모델 검증

황대윤

- 분석용 데이터 수집
- 데이터 전처리 및 EDA
- 데이터 모델링 학습 및 분석
- AI 분류 결과 검색 시각화 프론트/백엔드 구현
- 데이터 학습 모델 검증

김호현

- 분석용 데이터 수집
- 데이터 전처리 및 EDA
- 분류 모델링 학습 및 분석
- 게시판 프론트/백엔드 구현
- 데이터 학습 모델 검증

03. 프로젝트 수행 절차 및 방법

1. 프로젝트 수행 절차
2. 프로젝트 일정 관리
3. 프로젝트 협업 관리

1. 프로젝트 수행 절차

사전기획

▫ 11/24(월) ~ 11/30(금)
주제 선정 / 데이터 수집/
기획안 작성

데이터 및 웹 구성

▫ 12/1(월) ~ 12/4(목)
데이터 취합, 전처리,
EDA / 웹 프론트, 백엔드
구성

모델링 및 웹 구현

▫ 12/5(금) ~ 12/8(월)
ML 모델링 / 웹 프론트,
백엔드 구현(시각화)

모델링 검증/적용

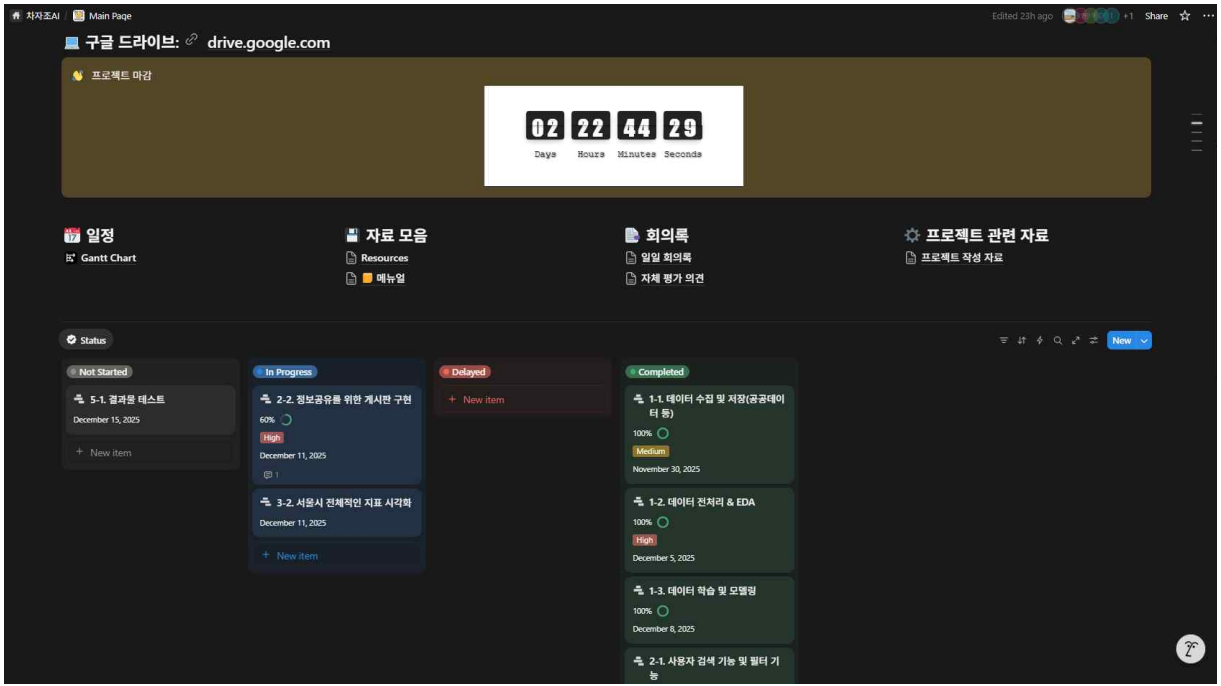
▫ 12/9(화) ~ 12/11(목)
ML 모델링 검증 및 웹에
적용 / 시각화

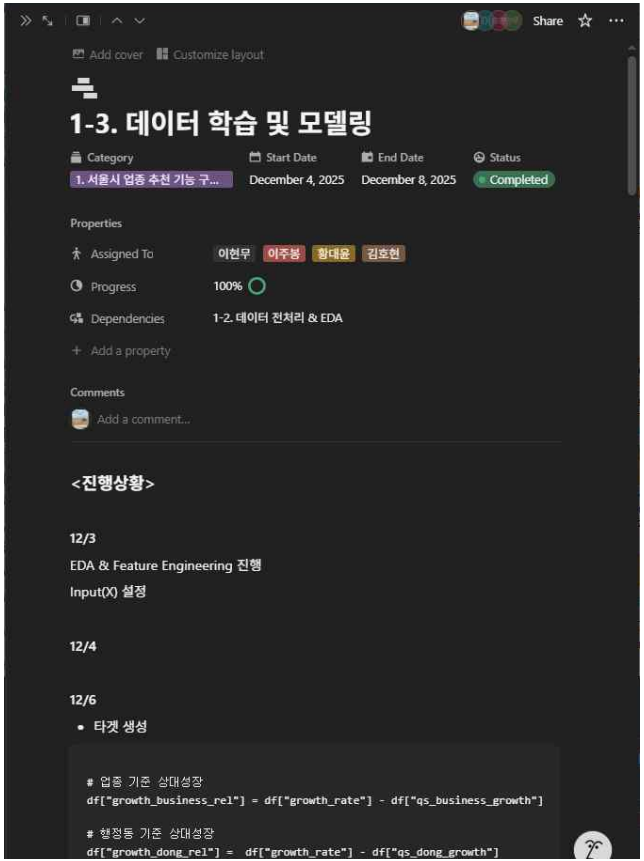
테스트 및 발표

▫ 12/12(금) ~ 12/15(월)
ML 모델 및 웹 테스트
프로젝트 리뷰 및 발표

3. 프로젝트 협업 관리

Notion





Gantt Chart | 결과보고서 작성(개인 별 작성) | 서울시 상권 추천 서비스 | 기획서+설계 - Google Drive | CODELAB-AI2-WBS-1팀.xlsx

notion.so/Gantt-Chart-2bd67e8266a18019af76ce950174077d

차자조AI / Main Page / Gantt Chart

Edited Dec 2 Share

Gantt Chart

Notion

Timeline | Status | Priority | Full Details | Status (1)

No date (1) New

November 2025 December

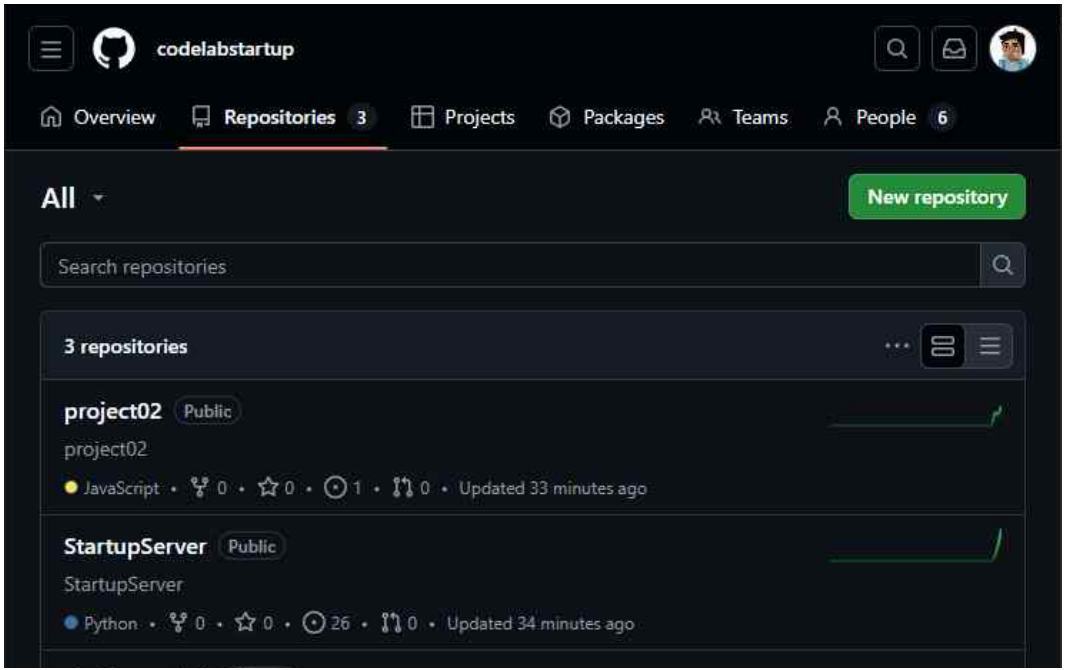
Manage in Calendar Month Today

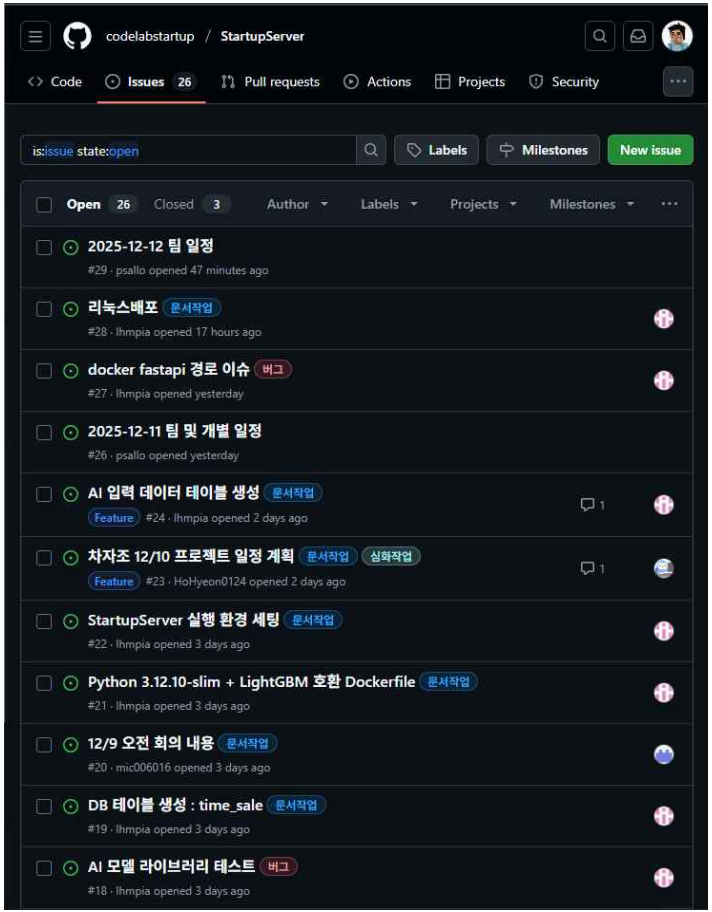
22 23 24 25 26 27 28 29 30 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 1 2

- 1. 서울시 업종 추천 기능 구현(AI모델링)
  - 1-1. 데이터 수집 및 저장(공공데이터 등) 100% Not Started
  - 1-2. 데이터 전처리 & EDA 100% Not Started
  - 1-3. 데이터 학습 및 모델링 100% Not Started
  - 1-4. 정확도 예측 및 추천 로직 구현 100% Not Started
  - 1-5. 추천 결과 시각화 50% On Hold
- + New
- 2. 검색 및 결과 정보 시각화 구현(프론트엔드)
  - 2-1. 사용자 검색 기능 및 필터 기능 100% Not Started
  - 2-2. 정보공유를 위한 게시판 구현 100% Not Started
- + New
- 3. 유형별 각종 지표에 대한 레포트 기능 구현
  - 3-1. 결과 차트(매출 추이, 인구 그래프 등) 시각화 100% Not Started
  - 3-2. 서울시 전체적인 지표 시각화 100% Not Started
- + New
- 4. API 서버 구현(백엔드)
  - 4-1. 추천 결과 제공 REST API 개발 100% Not Started
  - 4-2. 시각화를 위한 DB설계 및 구현 100% Not Started
  - 4-3. 조회, 필터링, 파라미터 기반 응답 구조 설계 100% Not Started
- + New
- 5. 테스트 및 발표준비
  - 5-1. 결과물 테스트 Not Started

3. 프로젝트 협업 관리

GitHub



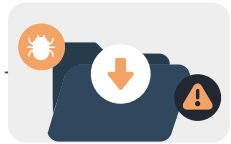


04. 프로젝트 수행 경과

AI 모델링을 위한 Data EDA

AI 모델 학습 및 검증

프론트엔드 구성



차트, 시각화 및 게시판

백엔드 구성

DB, FastAPI 및 AI모델 적용

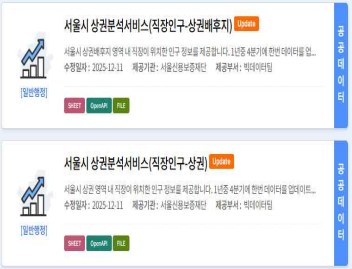
분석용 데이터 수집

분석용 데이터 수집 과정

서울 열린데이터 광장

상권 매출 데이터

서울 열린 데이터 광장에서 분기, 업종별 매출 정보 수집

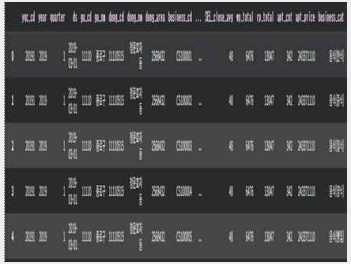


유동, 직장, 상주 인구

시간대, 요일, 연령대별 인구 이동 데이터를 서울 열린 데이터 광장에서 확보

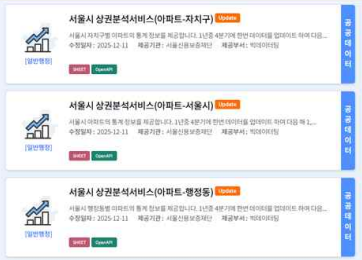
점포 상권 정보

점포수, 유사 업종 수, 상권 경계 등 상권 특성 데이터 수집



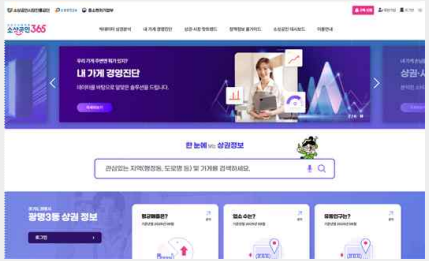
행정동, 업종 코드

Dong_cd, business_cd 등 공통 코드 테이블로 기준 정보 정리



인프라, 교통, 아파트

지하철, 버스 정류장, 교통량, 아파트 단지 데이터 수집



레퍼런스 사이트

- 서울시 상권분석 서비스
  https://golmok.seoul.go.kr/main.do
- 소상공인365
  https://bigdata.sbiz.or.kr

데이터 전처리 및 EDA(1)

```
# 데이터 정렬
df = df.sort_values(['dong_cd','business_cd','yqc_cd']).reset_index(drop=True)
df.head(10)
# 성장률(pct_change), rolling feature는 "시간 흐름" 순서가 중요
# yqc_code = 분기 코드, dong_cd = 행정동, business_cd = 업종 코드
# 동 → 업종 → 분기 순서로 시간 흐름을 맞춰 정렬
```

데이터 정렬 및 병합

dong_cd(행정동)·business_cd(업종)·yqc_cd(분기) 순으로 정렬해서
같은 동·업종 안에서 시간이 1분기→2분기→3분기... 순서대로 이어지도록 정리

이렇게 정렬해 두어야 pct_change()나 rolling() 같은 함수가
이전 분기와의 관계, 최근 3분기의 패턴을 정확하게 계산할 수 있다.

```
# 성장률 계산
df["qs_growth"] = df.groupby(["dong_cd", "business_nm"])["qs_total"].pct_change()
df.head()

# 동 + 업종 내부에서 해당 분기 매출이 이전 분기 대비 얼마나 증가/감소했는지 (%)를 계산한 것
# ex) -0.1 → -10% 감소, 0.3 → 30% 증가
```

성장률 계산 (qs_growth)

groupby(['dong_cd','business_nm'])['qs_total'].pct_change()를 이용해 **직전 분기 대비 매출이 몇 % 올랐는지/내렸는지**를 확인 가능

이 값은 단순한 매출 수준이 아니라
"최근에 성장 중인지, 둔화/감소 중인지"를 판단하는 핵심 지표로
이후 파생 변수와 타깃 정의의 기초가 된다.

데이터 전처리 및 EDA(2)

```
# 로그 성장률 계산 (성장률 기반)
df["log_growth"] = np.log1p(df["qs_growth"])
df["log_growth"]
```

스케일 조정 및 로그 변환

매출, 유동인구, 아파트 가격처럼 **값의 범위가 크고 치우친 지표**는 np.log1p()로 로그 변환하여 스케일을 줄이고 분포를 완화

이렇게 하면 극단적인 값(아웃라이어)의 영향을 줄이고,
모델이 다양한 피처를 비슷한 크기에서 학습할 수 있어 학습 안정성이 좋아짐.

```
# Rolling Feature (패턴 강화)
df["qs_roll3"] = df.groupby(["dong_cd", "business_cd"])["qs_total"].transform(lambda x: x.rolling(3).mean())
df["qs_vol"]   = df.groupby(["dong_cd", "business_cd"])["qs_total"].transform(lambda x: x.rolling(3).std())
df["qs_diff"]  = df.groupby(["dong_cd", "business_cd"])["qs_total"].transform(lambda x: x.diff())
```

파생 변수 생성

qs_roll3, qs_vol, qs_diff는 같은 동·업종 안에서
최근 3분기의 **평균 수준, 변동성, 직전 분기 대비 증감액**을 나타내는 변수.

단순히 한 분기 값만 보는 것이 아니라
"최근 추세가 우상향인지, 들쭉날쭉한지, 급락/급등이 있었는지" 같은
시간 패턴을 모델이 더 잘 인식할 수 있게 해준다.

데이터 전처리 및 EDA(3)

```python
# 업종별 Robust Scaling (업종별 정규화 작업)
def robust_norm(x):
    x_no_nan = x.dropna()
    if len(x_no_nan)==0:
        return pd.Series([np.nan]*len(x), index=x.index)
    med = x_no_nan.median()
    mad = np.median(np.abs(x_no_nan - med))
    return (x - med) / (mad + 1e-6)

df["growth_norm"] = df.groupby("business_cd")["log_growth"].transform(robust_norm)
df["growth_norm"]
```

업종별 정규화 작업

업종마다 성장률의 스케일이 다르기 때문에
business_cd별로 log_growth에 robust_norm()을 적용해 정규화.
중앙값과 MAD(중앙 절대 편차)를 이용하는 방식이라
**극단값에 덜 민감하고, 업종 간 수준 차이를 제거한 "상대 성장 지표"를 만들 수 있다
→ 결과 컬럼이 growth_norm.

```python
# 동 단위 상대 순위 기반 라벨 생성(추천/보통/비추천)

df["rank_pct"] = df.groupby(["dong_cd", "yqc_cd"])["growth_score"].rank(pct=True)
df["rank_pct"]
```

```python
df["target"] = np.where(
    df["rank_pct"] >= 0.80, 2,     # 추천
    np.where(df["rank_pct"] <= 0.20, 0, 1)   # 비추천 , 보통
)
df["target"]
```

타깃 정의 기준 탐색

- rank_pct는 동일한 동·분기 안에서 growth_score가 **상대적으로 몇 번째(상위 몇 %)에 위치하는지**를 나타내는 백분위 값.
- rank_pct ≥ 0.80은 상위 20% 상권(추천), ≤ 0.20은 하위 20% 상권(비추천), 그 사이를 보통으로 설정해 **추천/보통/비추천 3등급 타깃**을 만듬
- 이렇게 하면 절대값 기준이 아니라 **동·분기 내 상대적인 경쟁력**을 반영한 라벨링이 가능함

분류 모델링 학습 및 분석(1)

```
df["target"] = np.where(
    df["rank_pct"] >= 0.80, 2,      # 추천
    np.where(df["rank_pct"] <= 0.20, 0, 1)   # 비추천 , 보통
)
df["target"]
```

타깃 정의

앞에서 만든 target 컬럼은
0=비추천(하위 20%), 1=보통, 2=추천(상위 20%)을 의미하는 3분류 라벨이다.
이 타깃을 통해 모델이 **어떤 상권이 성장 가능성이 높은지(추천)
어떤 상권은 주의가 필요한지(비추천)**를 직접 예측하도록 학습시켰다.

```
model = LGBMClassifier(
    n_estimators=1300,
    learning_rate=0.05,
    max_depth=-1,
    num_leaves=64,
    class_weight='balanced',
    objective='multiclass',
    num_class=3,
    n_jobs=-1
)

model.fit(
    X_train,
    y_train,
    eval_set=(X_test, y_test)
)

pred = model.predict(X_test)
print(classification_report(y_test, pred))

model
```

모델 학습 선정 (LightGBM)

여러 트리 기반 모델 중에서 **학습 속도와 성능, 해석 가능성**을 고려해
LightGBM을 선택
num_leaves, max_depth, learning_rate, class_weight='balanced' 등
하이퍼파라미터를 조정하면서 Validation 성능(F1 점수)을 모니터링해
과적합을 줄이면서도 3개 클래스 모두를 잘 맞추는 조합 확인

분류 모델링 학습 및 분석(2)

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    stratify=y,          # 클래스 불균형이면 꼭 지정
    random_state=42
)
```

데이터 분할

전체 데이터를 **학습용(train)** 과 **테스트용(test)** 으로 나누는 함수
클래스 불균형(어떤 라벨이 훨씬 적은 경우)이 있을 때
stratify를 안 쓰면, **테스트 세트에 어떤 클래스가 거의 안 들어가 버리는 상황이 생길 가능성이 있어서 사용**

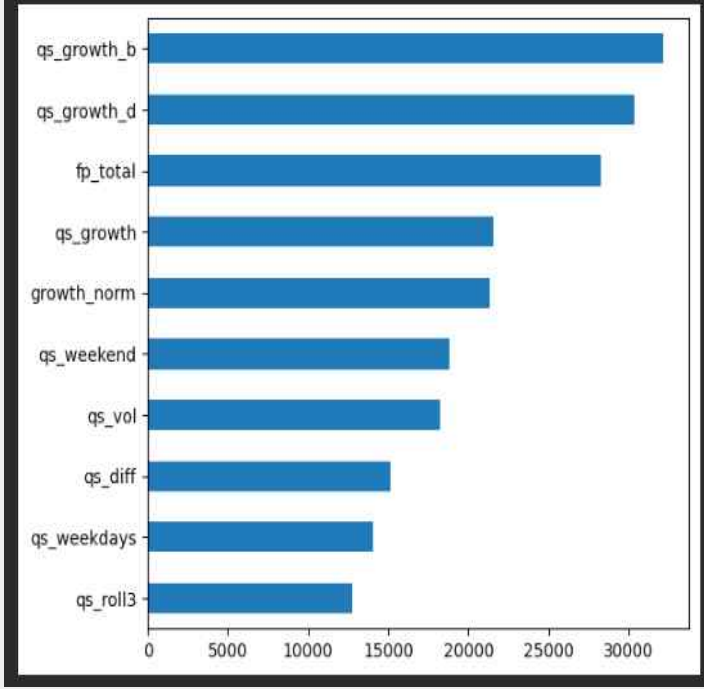
```
# 업종별 Robust Scaling (업종별 정규화 작업)
def robust_norm(x):
    x_no_nan = x.dropna()
    if len(x_no_nan)==0:
        return pd.Series([np.nan]*len(x), index=x.index)
    med = x_no_nan.median()
    mad = np.median(np.abs(x_no_nan - med))
    return (x - med) / (mad + 1e-6)

df["growth_norm"] = df.groupby("business_cd")["log_growth"].transform(robust_norm)
df["growth_norm"]
```

학습 및 튜닝

LightGBM 학습 시,
learning_rate, n_estimators, max_depth, min_child_samples 등의 값을 바꿔가며
Validation F1을 기준으로 여러 번 실험
규제 계수와 조기 종료(early stopping)를 함께 사용해
학습 데이터에는 잘 맞지만 테스트에는 성능이 떨어지는 과적합 상황을 피하는 것이 목표

분류 모델링 학습 및 분석(3)

```
fi = pd.Series(model.feature_importances_, index=feature_cols).sort_values(ascending=True)
plt.figure(figsize=(6,5))
fi.tail(10).plot(kind="barh")
plt.tight_layout()
```



분석 결과

Feature importance 결과에서
qs_growth_b·qs_growth_d·fp_total·qs_growth·growth_norm 등이
상위에 나타났고, 이는 **매출 성장률·상대 성장 점수·유동인구**가
성장형 상권을 구분하는 데 중요한 역할을 한다는 의미

또한 qs_weekend·qs_vol·qs_diff 같은 변수는
주말 매출 비중과 매출 변동성, 최근 증감 추세가
추천/비추천 상권을 나누는 보조 지표로 작용함을 보여줌.

이 결과를 통해,
"**꾸준한 매출 성장 + 유동인구 증가** 의 요인이
실제로 모델이 추천하는 상권이라는 해석"까지 연결

데이터 모델 검증 (LightGBM)

```
print("===== LightGBM 정확도 =====")
print(f"Accuracy: {accuracy_score(y_test, pred)*100:.2f}%\n")
print(classification_report(y_test, pred))
print(confusion_matrix(y_test, pred))
```

```
===== LightGBM 정확도 =====
Accuracy: 88.23%

              precision    recall  f1-score   support

           0       0.79      0.92      0.85     13822
           1       0.94      0.86      0.90     43807
           2       0.83      0.92      0.87     15658

    accuracy                           0.88     73287
   macro avg       0.85      0.90      0.87     73287
weighted avg       0.89      0.88      0.88     73287

[[12754  1068     0]
 [ 3348 37530  2929]
 [    0  1284 14374]]
```

앞 슬라이드의 Feature Importance 결과를 함께 해석한 결과,
매출 성장률·상대 성장 점수(예: qs_growth, qs_growth_b, qs_growth_d, growth_norm)와
최근 3분기 패턴(qs_roll3, qs_vol, qs_diff),
점포·경쟁·유동인구 관련 지표(store_cnt, similar_store_cnt, fp_total, qs_weekdays, qs_weekend) 가
성장형 상권을 구분하는 핵심 피처로 작용함을 확인

이를 바탕으로

- **성장률과 유동 수요가 꾸준히 증가하는 상권 → 투자·창업에 유리한 추천 상권**
- **성장률이 낮거나 변동성이 크고 경쟁 점포가 많은 상권 → 폐점·위험 상권으로 경고**

와 같이, 모델 결과를 **서비스 관점의 정책·운영 전략으로 해석**할 수 있음을 정리

단, 정확도가 88.23%로 높게 나왔지만, 타겟과 피처를 동일하게 잡아서 테스트를 진행
→ 이렇게 되면 정확도가 높게 나올 수 밖에 없기 때문에
이후 다른 방향으로 테스트 진행

데이터 모델 검증 (Catboost)

```python
print("===== MODEL ACCURACY =====")
print(f"Accuracy: {acc*100:.2f}%\n")

print("===== Classification Report =====")
print(classification_report(y_test, pred))

print("===== Confusion Matrix =====")
print(confusion_matrix(y_test, pred))
```

```
===== MODEL ACCURACY =====
Accuracy: 89.48%

===== Classification Report =====
              precision    recall  f1-score   support

           0       0.86      0.86      0.86     13822
           1       0.91      0.92      0.91     43807
           2       0.89      0.86      0.88     15658

    accuracy                           0.89     73287
   macro avg       0.89      0.88      0.88     73287
weighted avg       0.89      0.89      0.89     73287

===== Confusion Matrix =====
[[11854  1968     0]
 [ 1920 40312  1575]
 [    3  2241 13414]]
```

CatBoost 모델은
비추천·보통·추천 세 그룹 모두를 비교적 균형 있게 예측

Confusion Matrix 구조상

- 비추천 ↔ 추천 극단 상권은 잘 구분
- 다만 '애매한 중간 상권'은 위·아래 단계와 혼동되는 비율이 존재

서비스 관점에서

- **보통이지만 추천으로 자주 오분류되는 상권** → 잠재 성장 후보,
- **보통인데 추천으로 자주 오분류되는 상권** → 관리·경고가 필요한 상권
  으로 해석할 수 있어,
  단순 점수 제공을 넘어 **리스크/기회 상권을 구분하는 설명 가능한 추천 엔진**으로 활용 가능함

단, 정확도가 89.18%로 높게 나왔지만, 타겟과 피처를 동일하게 잡아서 테스트를 진행
→ 이렇게 되면 정확도가 높게 나올 수 밖에 없기 때문에
이후 다른 방향으로 테스트 진행

LightGBM vs CatBoost 모델 비교

기준	train _test_split(test_size=0.2, stratify=y) 로 같은 학습/ 검증데이터셋을 사용하며, 평가 기준은 Accuarcy, Marco F1, Confusion Matrix이다.

[LIghtGBM 모델]

N_setimators = 1300, learning_rate=0.05, num_leaves= 64, class_weight='balanced'로 설정하여 3분류(num_class) 모델 학습

동일한 14개 피처 (qs_growth, qs_roll3, qs_vol, store_cnt, fp_total 등) 사용하여 X_train, X_test를 구성하고 y는 동,분기 내 상대 순위로 만드는 0,1,2 타깃을 사용)

코드 실행 결과 기준으로 Accuracy는 88.23%, Marco F1 값은 0.87 수준으로, 세 클래스 모두 비교적 균형 잡힌 성능을 보임

장점은 학습 속도가 빠르고, 하이퍼파라미터를 조정하며 여러 실험을 돌리기 좋다는 점이며, 반면에 카테고리형 피처 처리나 복잡한 상호작용을 자동으로 잡는데는 Catboost에 비해 다소 제한적임

상권 추천 서비스에서 기본 베이스 라인 모델로 활용하기 적합하며, 추후 다른 알고리즘과 성능, 설명력 비교를 위한 기준 모델 역할을 함.

[Catboost 모델]

Iterations=1200, depth=8, learning_rate=0.02로 설정하고, loss_function='MultiClass', eval_metric='MultiClass'로 3분류 문제 학습

LightGBM과 동일하게 전처리된 df_train[feature_cols]와 0,1,2 타깃을 사용하여 train_test_split(test_size=0.2, stratify=y)로 학습/검증 데이터 구성

코드 실행 기준으로 Accuracy는 (89.48%), Marco F1은 (0.89) 수준으로, LightGBM과 유사하거나 약간 상회하는 성능을 보임

장점은 파라미터를 많이 손대지 않아도 안정적인 성능이 나오고, 복잡한 비선형 패턴을 잘 잡는다는 점이며, 단점은 학습시간이 상대적으로 길고 모델 크기가 다소 커질 수 있다는 점

성능과 안정성을 고려했을 때 최종 서비스 후보 모델로 사용 가능하며, 추후 Feature Importance, SHAP 분석을 통해 상권 성장 요인 설명하는데 활용

정보 공유 게시판 (익명 기반) 구현 과정

1 게시판 글 목록 페이지 구현

마운트 시점에 GET /board 호출 → 전체 게시글 배열을 받아와 상태로 저장하고, MUI Pagination으로 페이지 이동 기능 구현

2 게시판 상세 페이지 구현

URL 파라미터 id를 이용해 **GET /board/{id} 호출**, DB에서 조회수(ip_view_count)를 +1 증가시킨 뒤 제목·작성자·내용을 상세 페이지에 표시

3 게시판 글 작성 페이지 구현

title / writer / content / password를 React 상태로 관리하고, **폼 전송 시 POST /board/write**로 JSON 데이터를 전송하여 새 글을 등록, 성공 시 목록 페이지로 리다이렉트

4 게시판 글 수정 페이지 구현

상세 페이지에서 "수정" 버튼 클릭 시 기존 글 내용을 입력폼에 바인딩하고, **비밀번호를 함께 보내 PUT /board/{id}** 호출 → 비밀번호가 일치할 때만 제목·내용 업데이트

5 게시판 글 삭제 페이지 구현

삭제 버튼 클릭 시 비밀번호를 입력 받은 뒤 DELETE /board/{id} 요청

- "비밀번호가 맞으면 "삭제 완료" 메시지 후 목록으로 이동"
- 틀리거나 글이 없으면 403 에러 를 받아 사용자 에게 안내 문구 출력

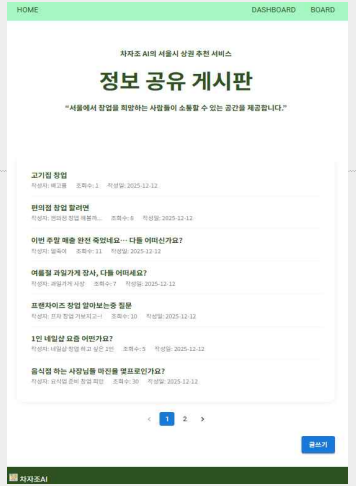
정보 공유 게시판 - 페이지별 기능 정리

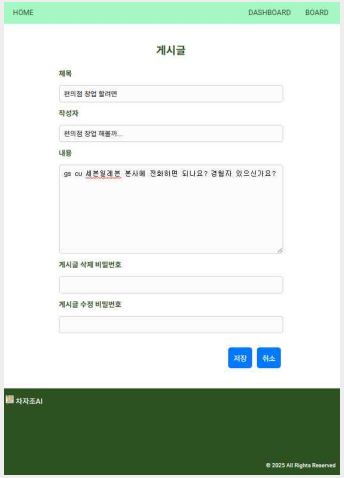
게시글 글 작성 페이지

제목, 작성자, 내용, 비밀번호 네가지 필드를 입력받아 새 글을 등록하는 화면

게시판 목록 페이지

백엔드 GET / board 응답을 기반으로 최신 글이 위로 오도록 정렬해 보여주는 화면



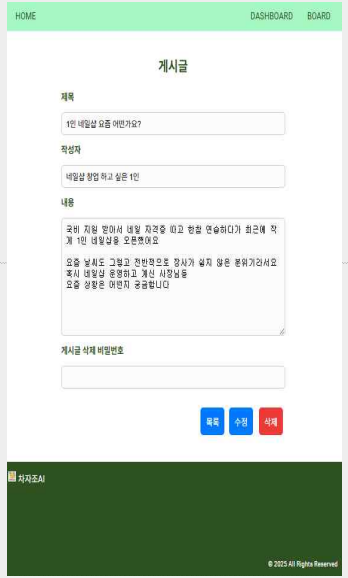


게시판 수정 페이지

상세 화면에서 "수정" 버튼을 눌렀을 때, 기존 제목, 내용을 그대로 불러와 편집 할 수 있는 화면

게시글 글 상세 페이지

선택한 게시글의 전체 내용을 확인하는 화면



목록, 작성, 상세, 수정 페이지 구현

DB 테이블 분석 및 DB 모듈 구현(dbt_ip.py) (1)

```python
from db.db_connection import get_connection

MYSQL_TABLE = "insight_post"

def insight_ps():
  try:
    con = get_connection()
    cursor = con.cursor()
    sql = f"""
        SELECT ip_id, ip_title, ip_content, ip_writer,
               ip_view_count, ip_created_at
        FROM {MYSQL_TABLE}
        ORDER BY ip_id DESC
    """
    cursor.execute(sql)
    rows = cursor.fetchall()
    columns = [col[0] for col in cursor.description]
    result_ps = [dict(zip(columns,row)) for row in rows]
    return result_ps
  except Exception as e:
    print("게시판 글 조회 오류:", e)
    return[]
  finally:
    if con:
      con.close()
```

- 기존 ERD에 있던 insight_post 테이블을 이용해 게시판 구조 확정
  - ip_id, ip_title, ip_content, ip_writer, ip_view_count, ip_created_at 등 사용
  - 익명 게시판 기능을 위해 ip_pw 컬럼을 추가하여 **비밀번호 저장**
- DB 접근 모듈 dbt_ip.py 작성
  - insight_ps()
    - SELECT ... FROM insight_post ORDER BY ip_id DESC
    - 게시판 목록을 **dict 리스트 형태**로 반환

DB 테이블 분석 및 DB 모듈 구현(dbt_ip.py) (2)

```python
def insight_ps_one(ip_id):
  try:
    con = get_connection()
    cursor = con.cursor()

    # 1) 조회수 1 증가
    update_sql = f"""
        UPDATE {MYSQL_TABLE}
        SET ip_view_count = ip_view_count + 1
        WHERE ip_id = %s
    """
    cursor.execute(update_sql, (ip_id,))
    con.commit()   # UPDATE 했으면 커밋

    # 2) 증가된 값 포함해서 다시 조회
    sql = f"""
        SELECT ip_id, ip_title, ip_content, ip_writer,
               ip_view_count, ip_created_at
        FROM {MYSQL_TABLE}
        WHERE ip_id = %s
    """
    cursor.execute(sql, (ip_id,))
    row = cursor.fetchone()

    if not row:
        return None

    columns = [col[0] for col in cursor.description]
    result_ps = dict(zip(columns,row))
    return result_ps

  except Exception as e:
    print("게시글 조회 (+ 조회수 증가) 오류:", e)
    return None

  finally:
    if con:
      con.close()
```

- insight_ps_one(ip_id)
  - 조회 시 ip_view_count = ip_view_count + 1로 **조회수 1 증가 후 단일 게시글 조회**

DB 테이블 분석 및 DB 모듈 구현(dbt_ip.py) (3)

```python
# write 글쓰기 함수
def insight_ps_write(title, writer, content, password):
    try:
        con = get_connection()
        cursor = con.cursor()

        sql = f"""
            INSERT INTO {MYSQL_TABLE}
                (ip_title, ip_content, ip_writer, ip_pw)
            VALUES (%s, %s, %s, %s)
        """
        # 조회수는 처음 0으로, 나머지는 파라미터로 전달
        cursor.execute(sql, (title, content, writer, password))
        con.commit()

        # 방금 INSERT된 row의 PK (ip_id)
        return cursor.lastrowid

    except Exception as e:
        print("게시판 글 쓰기 오류:", e)
        return None

    finally:
        if con:
            con.close()
```

- insight_ps_write(title, writer, content, password)
  - 새 글 INSERT, cursor.lastrowid로 생성된 ip_id 반환

```python
# delete 게시글 삭제 함수
def insight_ps_delete(ip_id, password):
    con = None
    try:
        con = get_connection()
        cursor = con.cursor()

        sql = f"""
            DELETE FROM {MYSQL_TABLE}
            WHERE ip_id = %s AND ip_pw = %s
        """
        # 삭제 게시글 넘버, 비밀번호
        cursor.execute(sql, (ip_id, password))
        con.commit()

        # 삭제된 행 개수 반환 (0이면 삭제 안된 것)
        return cursor.rowcount

    except Exception as e:
        print("게시판 글 삭제 오류:", e)
        return 0

    finally:
        if con:
            con.close()
```

- insight_ps_delete(ip_id, password)
- WHERE ip_id = %s AND ip_pw = %s 조건으로 삭제, rowcount로 **삭제 성공/실패(비밀번호 오입력 포함)** 구분

DB 테이블 분석 및 DB 모듈 구현(dbt_ip.py) (4)

```python
# Update 게시글 수정 함수
def insight_ps_update(ip_id, title, content, password):
  try:
    con = get_connection()
    cursor = con.cursor()

    sql = f"""
      UPDATE {MYSQL_TABLE}
      SET ip_title = %s,
          ip_content = %s
      WHERE ip_id = %s
        AND ip_pw = %s
    """
    cursor.execute(sql,(title, content, ip_id, password))
    con.commit()

    # 영향 받은 row 수 (0이면 실패)
    return cursor.rowcount

  except Exception as e:
    print("게시판 글 수정 오류:", e)
    return 0

  finally:
      if con:
          con.close()
```

- insight_ps_update(ip_id, title, content, password)
  - 같은 조건으로 UPDATE, 수정된 행 수로 성공 여부 판단

FastAPI 게시판 라우터 구현(board.py) (1)

```python
from fastapi import APIRouter, HTTPException, Request
from db.dbt_ip import insight_ps, insight_ps_write, insight_ps_one, insight_ps_delete, insight_ps_update

router = APIRouter()

@router.get("/")
def select_bd():
  bd_list = insight_ps()
  print("게시판 리스트;", bd_list)
  return bd_list

@router.get("/{ip_id}")
def select_bd_one(ip_id:int):
  bd_one = insight_ps_one(ip_id)
  print("게시판 글 상세보기;", bd_one)
  return bd_one
```

- APIRouter()를 이용해 /board 관련 라우터 분리
- 구현한 엔드포인트
  - GET /board/ → select_bd()
    - insight_ps()를 호출하여 **전체 게시글 목록 JSON 반환**
  - GET /board/{ip_id} → select_bd_one()
    - insight_ps_one(ip_id) 호출, 조회수 증가 포함 단일 글 반환

```python
@router.post("/write")
async def write_bd_one(request: Request):
    """
    게시글 작성: 프론트에서 보내준 JSON 받아서 DB에 INSERT
    """
    data = await request.json()

    title = data.get("title")
    writer = data.get("writer")      # 프론트의 author -> DB ip_writer 로 매핑
    content = data.get("content")
    password = data.get("password")

    # 필수값 체크
    if not title or not writer or not content or not password:
        raise HTTPException(status_code=400, detail="필수 항목이 누락되었습니다.")

    new_id = insight_ps_write(title, writer, content, password)

    if not new_id:
        raise HTTPException(status_code=500, detail="게시글 저장에 실패했습니다.")

    return {"message": "등록 완료", "ip_id": new_id}
```

- POST /board/write → write_bd_one()
  1. Request.json()으로 title, writer, content, password 받음
  2. 필수값 누락 시 400 Bad Request, DB 오류 시 500 처리
  3. 성공 시 "등록 완료", ip_id 응답

FastAPI 게시판 라우터 구현(board.py) (2)

```python
@router.delete("/{ip_id}")
async def delete_post(ip_id: int, request: Request):
    # 1) 프론트에서 보낸 JSON(body)에서 password 꺼내기
    data = await request.json()
    password = data.get("password")

    if not password:
        raise HTTPException(status_code=400, detail="비밀번호가 필요합니다.")

    # 2) DB에서 삭제 시도
    deleted_rows = insight_ps_delete(ip_id, password)

    # DB 자체 에러
    if deleted_rows < 0:
        raise HTTPException(status_code=500, detail="삭제 처리 중 오류가 발생했습니다.
        ")

    # 3) 삭제 실패
    if deleted_rows == 0:
        # 글이 없거나 비밀번호가 틀린 경우
        raise HTTPException(
            status_code=403,
            detail="비밀번호가 일치하지 않거나 삭제된 글입니다."
        )

    return {"message": "삭제 완료"}
```

- DELETE /board/{ip_id} → delete_post()
  - body에서 비밀번호를 받아 insight_ps_delete 호출
  - 삭제 행 수에 따라 400(비밀번호 없음) / 403(비밀번호 불일치) / 500(서버 오류) 처리

```python
@router.put("/{ip_id}")
async def update_post(ip_id: int, request : Request):
    data = await request.json()

    title = data.get("title")
    content = data.get("content")
    password = data.get("password")

  # 1) 값이 비어 있으면 400
    if not title or not content or not password:
        raise HTTPException(status_code=400, detail="필수 항목이 누락되었습니다.")

  # 2) 실제 DB 수정
    updated_rows = insight_ps_update(ip_id, title, content,password)

    if updated_rows == 0:
    # 글이 없거나 비밀번호가 틀린 경우
        raise HTTPException(status_code=403, detail="비밀번호가 일치하지 않거나 글이
        존재하지 않습니다.")

    return {"message": "수정 완료"}
```

- PUT /board/{ip_id} → update_post()
  1. 수정용 title, content, password를 받아 insight_ps_update 호출
  2. 영향 행 수가 0이면 **비밀번호 불일치 또는 글 없음 → 403**

게시판 목록 페이지 구현(BoardPage.jsx) (1)

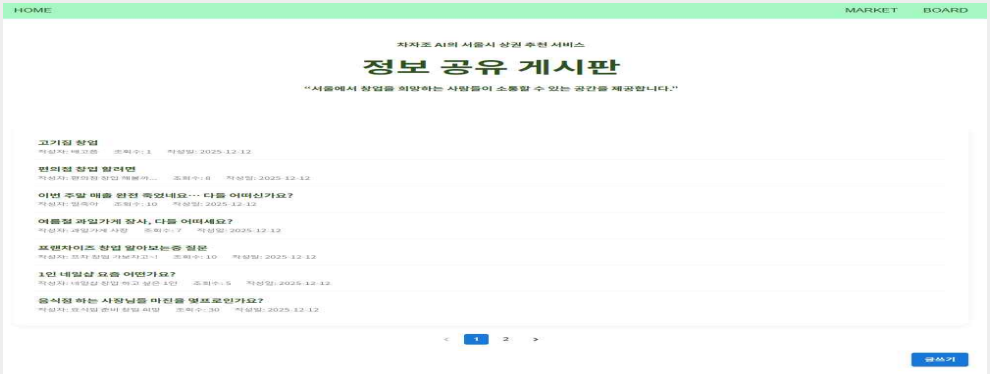
```
import { useEffect, useState } from "react"
import { Typography, Button, Pagination } from "@mui/material"
import styled from "@emotion/styled"
import axios from "axios"
import { useNavigate } from "react-router-dom"
import Stack from "@mui/material/Stack"

const API_BASE_URL = "http://localhost:8000" // ⚠ 백엔드 주소에 맞게 수정

export default function BoardPage() {
  const [posts, setPosts] = useState([])
  const [loading, setLoading] = useState(true)
  const [error, setError] = useState(null)
  const [page, setPage] = useState(1)
  const postsPerPage = 7
  const navigate = useNavigate()
```

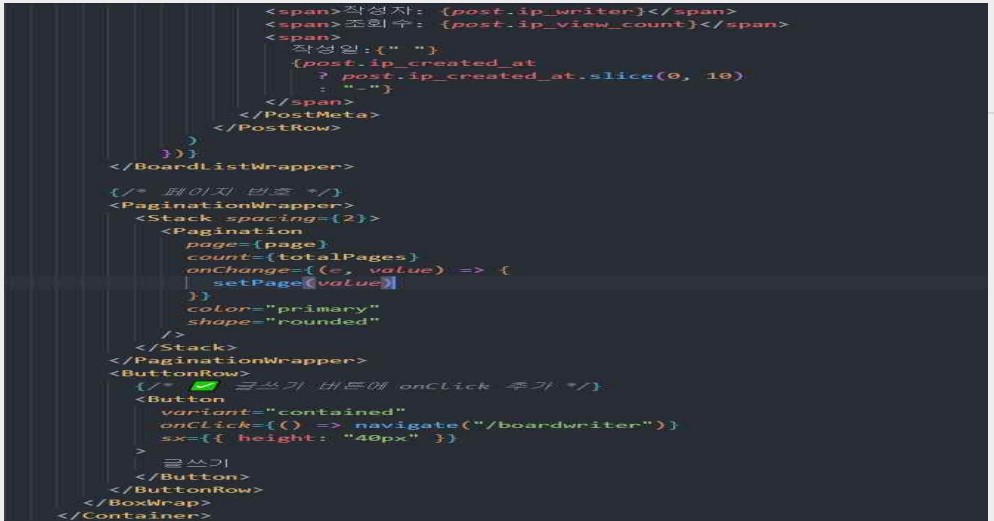
- 상태 관리
  1. posts, loading, error, page를 useState로 관리
  2. 한 페이지당 7개씩 보여주도록 postsPerPage = 7 설정

```
useEffect(() => {
  const fetchPosts = async () => {
    try {
      setLoading(true)
      setError(null)

      // 📌 서버 라우터:
      // @router.get("/") 이고, main.py 에서
      // app.include_router(board_router, prefix="/board") 라면:
      //   -> `${API_BASE_URL}/board`
      // prefix 없이 include 했다면:
      //   -> `${API_BASE_URL}/`
      const res = await axios.get(`${API_BASE_URL}/board`) // 필요하면 / 로 수정

      setPosts(res.data)
    } catch (err) {
      console.error("게시판 목록 조회 오류:", err)
      setError("게시판 목록을 불러오는 중 오류가 발생했습니다.")
    } finally {
      setLoading(false)
    }
  }

  fetchPosts()
}, [])
```

- 데이터 로딩
  1. 마운트 시 axios.get("${API_BASE_URL}/board") 호출
  2. 응답으로 받은 배열을 posts에 저장, 예외 발생 시 error 메시지 처리

게시판 목록 페이지 구현(BoardPage.jsx) (2)





```
                <span>작성자: {post.ip_writer}</span>
                <span>조회수: {post.ip_view_count}</span>
                <span>
                  작성일:{" "}
                  {post.ip_created_at
                    ? post.ip_created_at.slice(0, 10)
                    : "-"}
                </span>
              </PostMeta>
            </PostRow>
          )
        })}
      </BoardListWrapper>

      {/* 페이지 번호 */}
      <PaginationWrapper>
        <Stack spacing={2}>
          <Pagination
            page={page}
            count={totalPages}
            onChange={(e, value) => {
              setPage(value)
            }}
            color="primary"
            shape="rounded"
          />
        </Stack>
      </PaginationWrapper>
      <ButtonRow>
        {/* ✅ 글쓰기 버튼에 onClick 추가 */}
        <Button
          variant="contained"
          onClick={() => navigate("/boardwriter")}
          sx={{ height: "40px" }}
        >
          글쓰기
        </Button>
      </ButtonRow>
    </BoxWrap>
  </Container>
```

. UI 구성

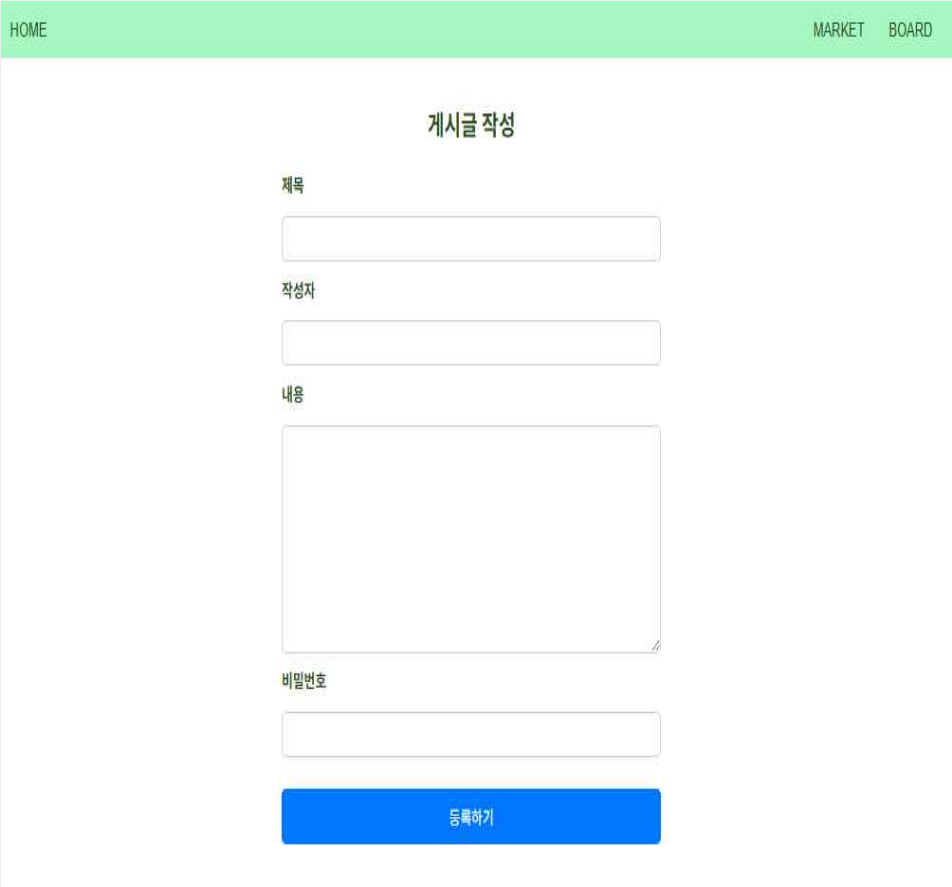
1. 상단에 **게시판 제목·설명 문구** 배치
2. posts를 map으로 렌더링하며, 현재 페이지 범위(startIndex~endIndex)만 보여줌
3. 각 행 클릭 시 navigate("/board/해당 ip_id")로 상세 페이지 이동
4. 하단에 Pagination 컴포넌트로 **페이지 이동 기능** 구현
5. 우측 하단 "글쓰기" 버튼 클릭 시 /boardwriter로 이동

게시판 작성 페이지 구현 (BoardWrite.jsx) (1)

```
import { useState } from "react"
import styled from "@emotion/styled"
import axios from "axios"
import { useNavigate } from "react-router-dom"

export default function BoardWrite() {
  const [form, setForm] = useState({
    title: "",
    author: "",
    content: "",
    password: "",
  })

  const navigate = useNavigate()

  const handleChange = (e) => {
    const { name, value } = e.target
    setForm((prev) => ({ ...prev, [name]: value }))
  }

  const handleSubmit = async (e) => {
    e.preventDefault()

    const payload = {
      title: form.title,
      writer: form.writer,
      content: form.content,
      password: form.password,
    }

    try {
      // ✅ 백엔드 라우터: POST /board/write 로 보냄
      await axios.post(`${import.meta.env.VITE_API_URL}/board/write`, payload)

      alert("게시글이 등록되었습니다!")
      setForm({ title: "", writer: "", content: "", password: "" })
      navigate("/board") // 목록으로 이동
    } catch (error) {
      console.error("게시글 등록 오류:", error)
      alert("서버와 통신할 수 없습니다.")
    }
  }
```

- 입력 폼 관리
  1. useState로 title, writer, content, password 필드를 가진 form 상태 생성
  2. handleChange에서 name, value로 동적으로 상태 업데이트
- 글 등록 처리
  1. onSubmit에서 e.preventDefault()로 기본 동작 막기
  2. payload = { title, writer, content, password } 생성 후 axios.post(VITE_API_URL + "/board/write", payload) 호출
  3. 성공 시 알림 출력 후 폼 초기화, /board로 이동
- 실패 시 콘솔 로그 + “서버와 통신할 수 없습니다.” 알림

게시판 작성 페이지 구현 (BoardWrite.jsx) (2)



. UI/UX

Emotion styled-components로 간단하지만 정돈된 폼 **UI** 구성

. 필수 입력(required) 속성을 통해 브라우저 기본 검증도 활용

게시판 상세 페이지 구현(BoardDetail.jsx)

```jsx
export default function BoardDetail() {
  useEffect(() => {
    const fetchPost = async () => {
        setError(null)

        const res = await axios.get(
          `${import.meta.env.VITE_API_URL}/board/${id}`
        )

        const data = res.data
        // ✅ 백엔드에서 반환하는 키 이름에 맞게 매핑
        setForm({
          title: data.ip_title ?? "",
          writer: data.ip_writer ?? "",
          content: data.ip_content ?? "",
          password: "", // 비밀번호는 보통 안 내려줌
        })
      } catch (err) {
        console.error("게시판 상세 조회 오류:", err)
        setError("게시글을 불러오는 중 오류가 발생했습니다.")
      } finally {
        setLoading(false)
      }
    }

    if (!id) return

    // StrictMode 때문에 두 번 실행되는 것을 방지하는 가드
    if (hasFetched.current) return
    hasFetched.current = true

    fetchPost()
  }, [id])

  if (loading) {
    return <Wrapper>게시글을 불러오는 중입니다...</Wrapper>
  }

  if (error) {
    return <Wrapper>{error}</Wrapper>
  }
```

- 데이터 로딩
  1. useParams로 URL의 id를 가져와 axios.get("/board/{id}") 호출
  2. 응답 데이터의 ip_title, ip_writer, ip_content를 form 상태에 저장
  3. loading·error 상태로 로딩 메시지 및 오류 메시지 표시
- useRef(hasFetched)로 React Strict Mode에서 **중복 호출 방지**

게시판 수정 페이지 구현(BoardDetail.jsx)

```jsx
export default function BoardDetail() {
  const handleEditStart = () => {
    // 현재 글 내용을 편집용 상태에 복사
    setEditForm({
      title: form.title,
      writer: form.writer,
      content: form.content,
    })
    setEditPassword("")
    setIsEditing(true)
  }

  const handleEditCancel = () => {
    setIsEditing(false)
  }

  const handleEditSave = async () => {
    if (!window.confirm("수정 내용을 저장하시겠습니까?")) return

    if (!editForm.title || !editForm.content) {
      alert("제목과 내용을 입력해주세요.")
      return
    }
    if (!editPassword) {
      alert("비밀번호를 입력해주세요.")
      return
    }

    try {
      await axios.put(`${import.meta.env.VITE_API_URL}/board/${id}`, {
        title: editForm.title,
        content: editForm.content,
        password: editPassword,
      })

      alert("수정되었습니다.")
```

```jsx
      // 상세 화면(state)도 수정된 내용으로 갱신
      setForm((prev) => ({
        ...prev,
        title: editForm.title,
        content: editForm.content,
      }))

      setIsEditing(false)
      setEditPassword("")
    } catch (err) {
      console.error("수정 오류", err)

      if (err.response && err.response.status === 403) {
        alert("비밀번호가 일치하지 않습니다.")
      } else if (err.response && err.response.status === 404) {
        alert("존재하지 않는 글입니다.")
      } else {
        alert("수정 중 오류가 발생했습니다.")
      }
    }
}
```

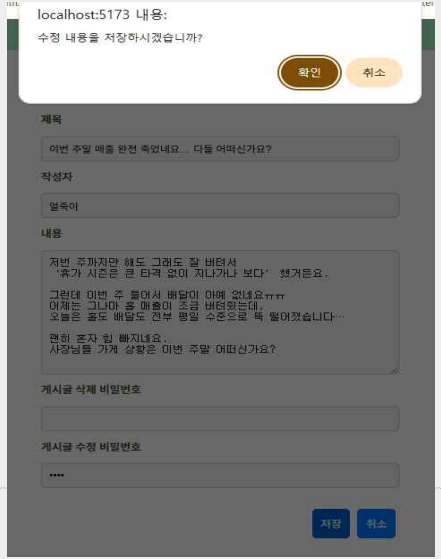
- 수정 기능
  1. "수정" 버튼 클릭 시 isEditing을 true로 바꾸고, 현재 내용을 editForm에 복사 → 입력 가능 상태로 변경
  2. "저장" 버튼에서 axios.put("/board/{id}", { title, content, password }) 호출
  3. 비밀번호 불일치 시 403 처리 → "비밀번호가 일치하지 않습니다." 알림
- 성공 시 화면 내용 갱신 후 편집 모드 종료

게시판 삭제 페이지 구현(BoardDetail.jsx)

```
const handleDelete = async () => {
  if (!window.confirm("정말 이 글을 삭제하시겠습니까?")) return

  if (!deletePassword) {
    alert("비밀번호를 입력하세요.")
    return
  }

  try {
    await axios.delete(`${import.meta.env.VITE_API_URL}/board/${id}`, {
      // axios의 DELETE 에서 body 보내는 법: data 속성 사용
      data: { password: deletePassword },
    })
    alert("삭제되었습니다.")
    navigate("/board")
  } catch (err) {
    console.error("삭제 오류", err)

    if (err.response && err.response.status === 403) {
      alert("비밀번호가 일치하지 않습니다.")
    } else if (err.response && err.response.status === 404) {
      alert("이미 삭제되었거나 존재하지 않는 글입니다.")
    } else {
      alert("삭제 중 오류가 발생했습니다.")
    }
  }
}
```

- 삭제 기능
  - 별도의 deletePassword 입력칸에서 비밀번호를 받고,
    axios.delete("/board/{id}", { data: { password } }) 호출
  - 성공 시 알림 후 /board로 이동
- 403/404 등 상태 코드에 따라 적절한 안내 메시지 출력

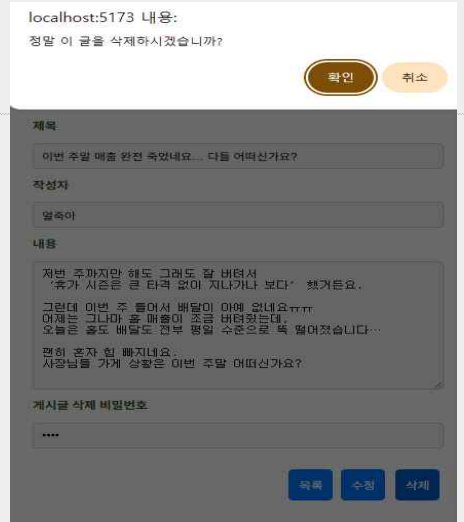
게시판 상세/수정/삭제 페이지 구현(BoardDetail.jsx)



- UI/UX

  읽기 모드일 때는 인풋 배경을 연한 회색으로 둬서 **읽기 전용 느낌** 강조

- 편집 모드에서만 onChange 허용, 버튼 라벨을 "수정/삭제 ↔ 저장/취소"로 토글
- 수정 비밀번호 입력 시 게시글 수정 가능 (alert 메시지: 수정 내용을 저장하시겠습니까?)
- 삭제 비밀번호 입력 시 게시글 삭제 가능 (alert 메시지: 정말 이 글을 삭제하시겠습니까?)



05. 자체 평가 의견

자체 평가 의견

프로젝트 결과물 완성도

상권분석에 대한 AI를 어떻게 정립해야 하는지 지식이 없는 상태에서 답을 찾아가면서 팀원과의 소통이 잘 이루어져서 결과물에 완성도가 생각보다는 잘 되었다고 생각

데이터 수집 → 전처리/EDA → 모델 학습 → FastAPI 연동 → 프론트 화면 구현까지 하나의 흐름으로 이어졌다는 점에서 완성도가 높다고 생각

잘한 부분과 아쉬운 부분

이번에 ai 모델 학습 머신러닝 과정을 2차 프로젝트에 도입함과 동시에 프론트&백엔드 구현까지 하려고 하니 생각 보다 많이 복잡했다는 생각

초기 기획에서 제시했던 상권 분석·추천 기능과 AI 리포트, 정보 공유 게시판까지 기본적인 핵심 기능은 모두 구현한 점은 잘했다고 생각

프로젝트 결과물의 개선점 및 보완점

서버 자동배포 기능 + REACT 자동 빌드 배포

현재 모델은 서울시 데이터를 기준으로 학습되어 있어, 다른 지역(경기·수도권 전체)으로 확장하기 위한 데이터 수집과 추가 학습이 필요

느낀점과 경험한 성과

이번 프로젝트를 통해 데이터 분석과 웹 서비스 개발이 따로 떨어진 일이 아니라, 하나의 흐름으로 연결된다는 것을 체감

docker를 이용하여 개발환경에서 배포까지 통합하려는 목적을 이룬 느낌이고, 호스트OS가 뭐든간에 실행할 수 있는 것을 확인

THANK YOU

Question?